

CORE POINT

Do not standardize two unrelated audio systems. Standardize one layered system with clear extension boundaries.

SLIDE 1

Why This Proposal Exists

Keep glTF on one audio path: simple assets stay simple, advanced assets gain layered capabilities instead of a competing model.

- KHR_audio_emitter already covers the direct asset path well: `audio[] -> sources[] -> emitters[]`.
- Advanced audio still needs processing graphs, explicit listeners, and environment semantics.
- The proposal extends the current foundation instead of replacing it.
- That reduces interoperability risk across tools, exporters, validators, and runtimes.

Issue: [KhronosGroup/glTF#2561](#)

Proposal repo: [rudybear/KHR_audio_proposals](#)

Runtime repo: [rudybear/AudioGraphJS](#)



SLIDE 2

Layered Hierarchy

Each extension owns one layer of meaning. That separation is the main architectural argument.

KHR_AUDIO_EMITTER

Content layer. Defines reusable audio payloads, playback sources, emitters, and scene or node bindings.

KHR_AUDIO_GRAPH

Signal-flow layer. Defines graph nodes, edges, source bindings, and emitter output bindings.

KHR_AUDIO_ENVIRONMENT

Scene-acoustics layer. Defines listener selection, environment selection, and room/reverb behavior.

```
KHR_audio_emitter
  audio[]
  sources[]
  emitters[]
  node.extensions.KHR_audio_emitter.emitters[]
  scene.extensions.KHR_audio_emitter.emitters[]
```

```
KHR_audio_graph
  graphs[]
  nodes[]
  connections[]
  inputs[] -> sources[]
  outputs[] -> emitters[]
```

```
KHR_audio_environment
  scenes[]
  listener
  environment
```

SEPARATION

Base = content binding. Graph = signal flow. Environment = scene acoustics. Review stays tractable because those concerns are not collapsed together.

Architecture first.



SLIDE 3**JSON Placement In glTF**

The structure stays familiar: root declarations for reusable audio assets, scene scope for listener and environment selection.

```
{
  "extensionsUsed": [
    "KHR_audio_emitter",
    "KHR_audio_graph",
    "KHR_audio_environment"
  ],
  "extensions": {
    "KHR_audio_emitter": {
      "audio": [],
      "sources": [],
      "emitters": []
    },
    "KHR_audio_graph": {
      "graphs": []
    }
  },
  "scenes": [
    {
      "extensions": {
        "KHR_audio_environment": {
          "listener": 0,
          "environment": 0
        },
        "KHR_audio_emitter": {
          "emitters": [0]
        }
      }
    }
  ]
}
```

ROOT

Reusable definitions live at the document root, which matches how glTF already treats many asset-level extension payloads.

SCENE

Scene extensions choose the active listener and environment without forcing scene acoustics into the emitter base object.

Document model remains
glTF-native.



WHY THIS MATTERS

It solves a real content problem while preserving the base extension as the lean interoperability layer.

SLIDE 4

Minimal Base Change

The base should change only where real authoring pressure exists.

- Change node-level emitter to emitters.
- Allow multiple emitters on one node without synthetic child nodes.
- Match scene-level array semantics already used by emitter bindings.
- Keep routing, listener, and environment semantics out of the base extension.

```
{
  "node": {
    "extensions": {
      "KHR_audio_emitter": {
        "emitters": [0, 2]
      }
    }
  }
}
```

Narrow base change.



SLIDE 5

Graph Schema Shape

The graph object is small enough to review directly: nodes, edges, source bindings, emitter bindings.

GRAPH OBJECT

graphs[] contains nodes[], connections[], inputs[], and outputs[].

```
{
  "name": "fx-chain",
  "nodes": [],
  "connections": [],
  "inputs": [],
  "outputs": []
}
```

SEMANTICS

inputs[] bind base sources[] into graph entry points. outputs[] bind graph results into base emitters[].

```
{
  "inputs": [{ "source": 0, "node": 0 }],
  "outputs": [{ "node": 2, "emitter": 0 }]
}
```

```
{
  "graphs": [
    {
      "name": "fx-chain",
      "nodes": [
        { "kind": "audiomixer", "params": {} },
        { "kind": "gain", "params": { "gain": 0.8 } },
        { "kind": "lowpass", "params": { "frequency": 8000 } }
      ],
      "connections": [
        { "from": { "node": 0 }, "to": { "node": 1 } },
        { "from": { "node": 1 }, "to": { "node": 2 } }
      ],
      "inputs": [
        { "source": 0, "node": 0 },
        { "source": 1, "node": 0 }
      ],
      "outputs": [
        { "node": 2, "emitter": 0 }
      ]
    }
  ]
}
```

INFORMATIVE NOTE

Multiple graph outputs can target the same emitter.

Implementations realize this as summing into a shared emitter input bus by default.

Schema and runtime align.



EVIDENCE

Reviewer confidence comes from observable behavior: parser, runtime, validation, and real glTF fixture coverage.

SLIDE 6

Working Runtime And Real Fixtures

AudioGraphJS now demonstrates the layered design with concrete glTF fixtures and runtime behavior.

- Layered parser and merged graph execution are in place.
- Listener and scene environment are applied during rendering.
- Emitter fan-in mixing is real runtime behavior, not just prose.
- glTF radians are converted correctly to Web Audio degrees for cone angles.
- Validation and tests run against actual layered glTF examples.

Fixtures used in testing

- simple-emitter-only.json
- spatial-with-environment.json
- same-emitter-multi-graph.json

Verified locally

- npm run build
- npm test
- npm run spec:validate:layered
- npm run example:run-graph -- examples/graphs-layered/same-emitter-multi-graph.json

[Proposal repo](#)

[Runtime repo](#)

[Execution primer: 7-agent-pipeline](#)

Implementation baseline exists.